

A arquitetura Transformer

Tokenização · Arquitecturas · Backpropagation

$$\begin{aligned} p(x) = & \\ & p(x_1) \cdot \\ & p(x_2 | x_1) \cdot \\ & p(x_3 | x_1, x_2) \\ & \dots \end{aligned}$$

Onde estávamos

Na semana passada:

- Modelar $p(x)$ é a ideia central de todo o generativo
- Uma Gaussiana falha — não captura a geometria do manifold
- A factorização autoregressiva decompõe o problema:

$$p(x) = p(x_1) \cdot p(x_2 | x_1) \cdot p(x_3 | x_1, x_2) \cdot \dots$$

Hoje:

A máquina que implementa esta ideia.

MLP $\rightarrow$ RNN $\rightarrow$ CNN $\rightarrow$ **Transformer**

Vamos CONSTRUIR um GPT

4.192

parâmetros — o nosso microGPT



mesma
arquitetura

1T+

parâmetros — GPT-4

- 6 exercícios · ~200 linhas de código total
- Zero bibliotecas — só Python puro
- Algoritmicamente idêntico ao GPT-2, GPT-3, GPT-4
- **A arquitetura é a mesma. A escala é diferente.**

Os 6 exercícios

#	Exercício	Conceito-chave
1	Autograd Engine	Gradientes, chain rule + tokenizer
2	Building Blocks	Linear, softmax, RMSNorm
3	Self-Attention	Q/K/V, scaled dot-product
4	Full GPT Model	Multi-head attention, residuals, forward pass
5	Training	Cross-entropy loss, SGD, Adam
6	Generation + Experiments	Sampling, ablações — o core da avaliação

Hoje recebem o mapa. E começam a construir — pelos alicerces (ex1 e ex2).

Teoria distribuída pelas 3 semanas

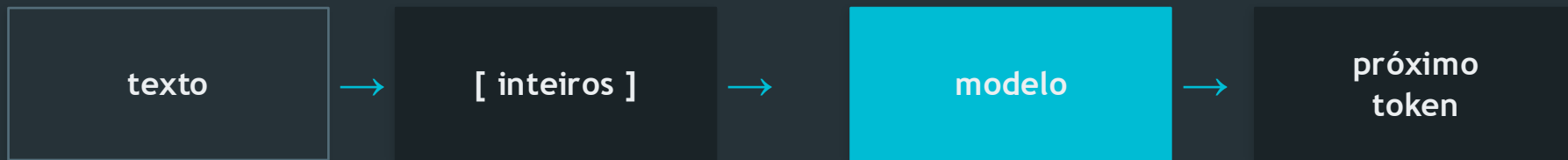
Semana	Teoria (~40 min)	Exercícios
2 · hoje	Tokenização + mapa de arquitecturas	ex1, ex2
3	Self-attention a fundo	ex3, ex4
4	Treino + ponte microGPT → GPT real	ex5, ex6

A teoria dá-vos o **mapa** — o que estão a construir e porquê.

Os exercícios dão-vos os **alicerces** — as peças que compõem esse mapa.

Como é que o modelo VÊ texto?

Um modelo não lê letras. Lê **números inteiros**.



O processo de converter texto em inteiros chama-se **tokenização**.

É o primeiro passo de qualquer LLM — e é o que vão implementar no exercício 1: construir o vocabulário, as funções encode e decode.

3 abordagens de tokenização

Abordagem	Exemplo: "playing"	Vocabulário	Sequência
Caracter	p, l, a, y, i, n, g	~100	Longa
Palavra	playing	~100.000+	Curta
Subword (BPE) ★	play, ing	~50.000	Média

Caracter

Simples, mas sequências longas — contexto gasto em letras.

Palavra

Vocabulário explode. Palavras novas ou gralhas → token desconhecido.

Subword (BPE)

O sweet spot. Vocabulário finito, palavras comuns inteiras, raras decompostas. Usado pelo GPT, LLaMA, Claude.

BPE em ação

DEMO LIVE

Byte Pair Encoding — fundir iterativamente os pares mais frequentes:

```
Passo 0: l o w   l o w e r   l o w e s t   (caracteres)
Passo 1: lo w   lo w e r   lo w e s t   (merge l+o)
Passo 2: low   low e r   low e s t   (merge lo+w)
:         'er' e 'est' aparecem em muitas palavras → fundidos cedo
Final:    low   low er   low est
```

Os subwords resultantes fazem sentido linguístico: "low", "er", "est".

tiktokenizer.vercel.app (~3 min) | Comparem GPT-2 vs GPT-4o no mesmo texto.

Experimentem: código Python, texto em PT, fórmulas químicas — cada tokenizer "vê" o texto de forma diferente.

Implicações da tokenização



Context window

Medida em tokens, não em palavras



Português $\approx$ 1.3× mais tokens

BPE treinado em corpora maioritariamente EN — o mesmo conteúdo em PT usa mais tokens



Custo de API

Tokens de output custam $\sim$ 3–4× mais que os de input (output é gerado token a token; input é processado em paralelo)



Artefactos da tokenização

Explicam comportamentos estranhos: dificuldade em contar letras, inverter palavras, aritmética simples

No microGPT: tokenização por caracter

O nosso vocabulário: **27 tokens** (26 letras + BOS = Beginning-of-Sequence)

```
uchars = ['a', 'b', 'c', ..., 'z']    # a=0, b=1, ..., z=25
BOS = 26                               # Beginning-of-Sequence
vocab_size = 27
```

```
encode("emma")      => [4, 12, 12, 0]
decode([4, 12, 12, 0]) => "emma"
```

É isto que vão implementar no exercício 1: vocabulário, encode e decode.

Cada character mapeia para um inteiro. O modelo nunca vê strings — só tensores.

THINK: qual é a loss de um modelo aleatório?

Se o modelo atribui probabilidade p ao token correcto: $\text{loss} = -\log(p)$

✦ THINK

$$p = 1$$

(certeza absoluta)

$$\text{loss} = 0$$

$$p = 1/27$$

(palpite aleatório)

$$\text{loss} = \log(27) \approx 3.30$$

$$p \rightarrow 0$$

(completamente errado)

$$\text{loss} \rightarrow \infty$$

Guardem o **3.30** — é o baseline de um modelo que não aprendeu nada.

Quando treinarem no ex5, a loss inicial deve ser ≈ 3.30 . Se for muito diferente, algo está errado.

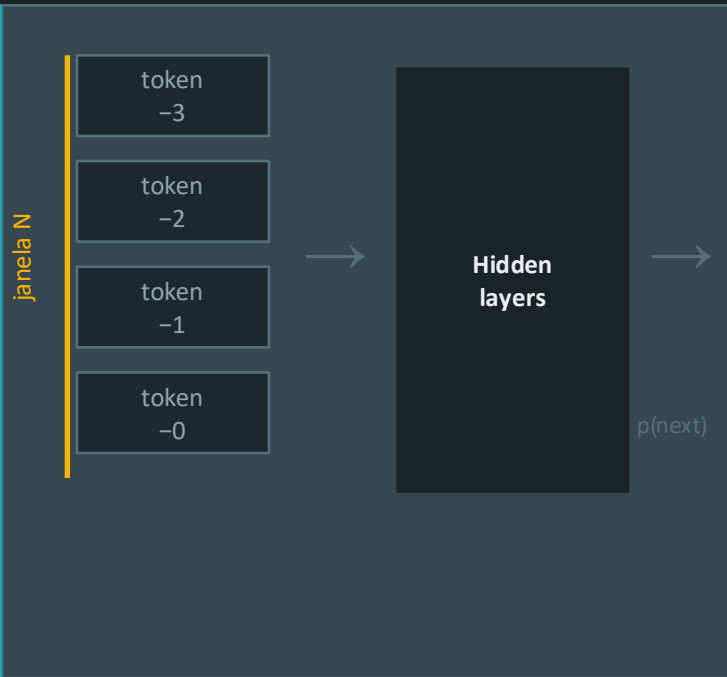
Precisamos de uma máquina

$$p(x) = p(x_1) \cdot p(x_2 | x_1) \cdot p(x_3 | x_1, x_2) \cdot \dots$$

Para estimar cada $p(x_t | x_{<t})$, precisamos de uma rede que receba os tokens anteriores e produza uma distribuição sobre o próximo.

A pergunta é: **como é que a rede "vê" os tokens anteriores?** Todos? Só os últimos N? Um resumo comprimido? A resposta define a arquitectura.

Tentativa 1: MLP



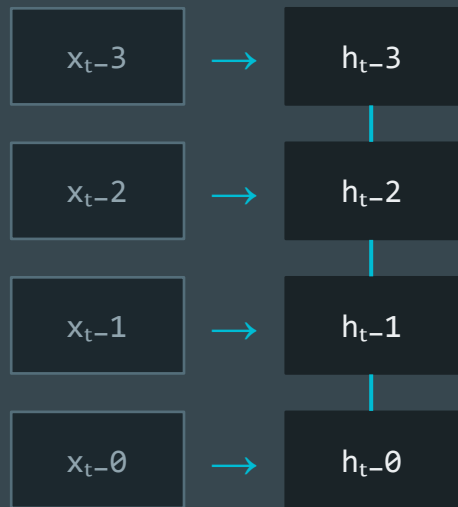
Vantagens

- ✓ Simples
- ✓ Paralelo

Limitação

- ✗ Janela fixa + pesos rígidos por posição — não aprende que token -5 pode ser mais relevante que token -1
- ✗ Sliding window não resolve: nunca vê dois tokens distantes juntos

Tentativa 2: RNN



$$h_t = f(h_{t-1}, x_t)$$

Vantagens

✓ Contexto ilimitado em teoria

Limitações

✗ Vanishing gradient — informação dos primeiros tokens desvanece

✗ Processamento sequencial — não paraleliza

Tentativa 3: CNN

Filtros locais aplicados em paralelo. Empilhar camadas aumenta o receptive field.



Paralelo, rápido



Receptive field limitado — precisa de muitas camadas para ver tokens distantes

(Arquitetura histórica — o Transformer superou-a.)

A solução:

Transformer

Cada token olha directamente para TODOS os anteriores

Acesso directo a qualquer token no contexto · Sem bottleneck sequencial · Paralelizável · Escalável

(A context window existe — mas por custo computacional, não por limitação arquitectural)

"Attention Is All You Need" · Vaswani et al., 2017

É isto que vão implementar.

Resumo: cada passo resolve o anterior

Arquitetura	Como condiciona	Limitação
MLP	Janela fixa · pesos fixos por posição	Não vê além de N · não foca no que importa
RNN	Hidden state sequencial	Vanishing gradient
CNN	Filtros locais empilhados	Receptive field limitado
Transformer ★	Atenção dinâmica sobre todos	Custo quadrático $O(n^2)$

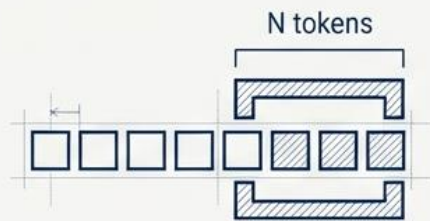
Todas modelam $p(x)$. A diferença é **como condicionam** nos tokens anteriores. MLP: pesos **fixos**.

Transformer: atenção **dinâmica**.

O custo quadrático do Transformer explica os limites da context window: duplicar o contexto quadruplica o custo de attention. A janela da MLP está na arquitetura; a do Transformer é uma escolha de engenharia.

A evolução do condicionamento em tokens anteriores

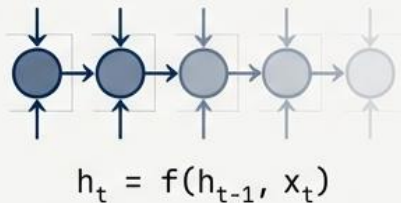
MLP



Janela fixa de N tokens.

Limitação: Não consegue ver além de N (esquece o início da frase).

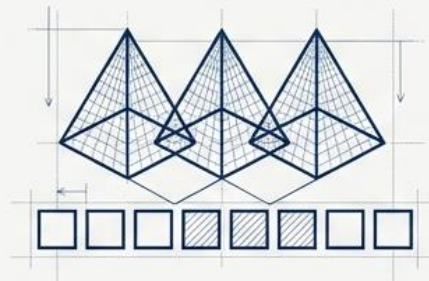
RNN



Hidden state sequencial.

Limitação: Vanishing gradient (a memória desvanece) e processamento lento não-paralelizável.

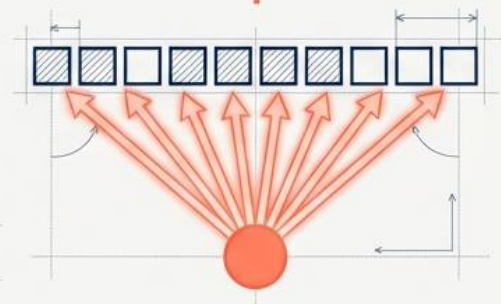
CNN



Filtros locais aplicados em paralelo.

Limitação: Receptive field limitado.

Transformer



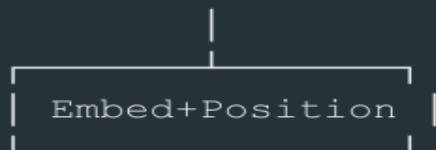
Attention. Cada token olha diretamente para TODOS os tokens anteriores em paralelo.

Trade-off: Custo computacional quadrático.

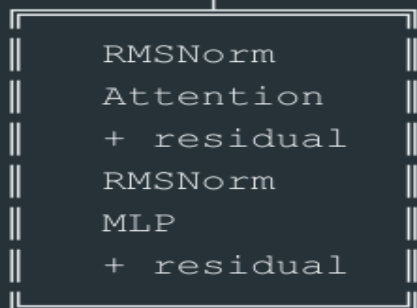
O mapa do Transformer (decoder-only)

"emma" → Tokenizer → [4, 12, 12, 0]

← ex1



← ex4



← ex2

← ex3

← ex4

← ex2

← ex2

← ex4

× N (N=2)

RMSNorm
Language Head

← ex2

← ex2

p(next token)

← ex5/ex6

Percorso bottom-up: 9 passos

1 Tokenizer

texto → inteiros (ex1)

2 Embedding

inteiros → vectores densos (ex4)

3 Positional encoding

injectar informação de ordem (ex4)

4 RMSNorm

normalizar antes de cada sub-camada (ex2)

5 Self-Attention

cada token olha para os anteriores (ex3)

6 Residual connection

atalho que estabiliza o treino (ex4)

7 MLP

transformação não-linear (ex2, linear + ReLU)

8 Language Head

projectar para vocabulário (ex2, linear + softmax)

9 Loss + Training

cross-entropy + optimizador (ex5)

Decoder-only vs outros

Variante	Exemplos	Direcção
Decoder-only ★	GPT, LLaMA, Claude	Esquerda → direita (causal)
Encoder-only	BERT	Bidirecional (masked)
Encoder-Decoder	T5, tradução	Encode input + decode output

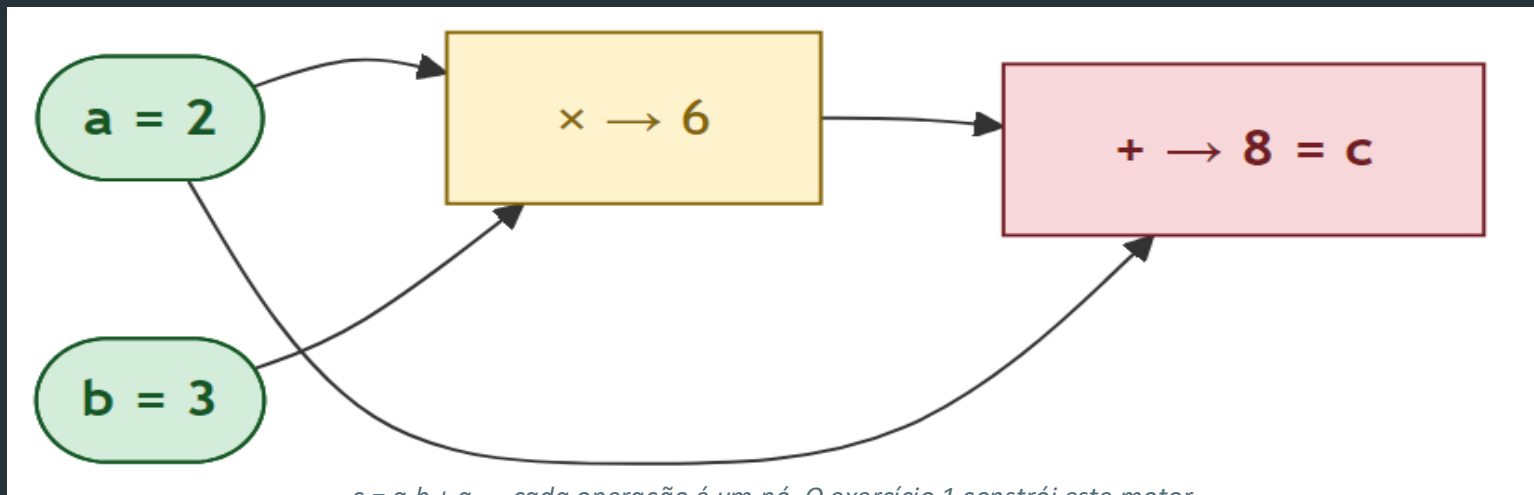
Neste curso: **decoder-only**. É a arquitectura dos LLMs modernos.
A diferença está na máscara de attention — verão isto no exercício 3.

Como é que o modelo aprende?

O mapa do Transformer mostra o forward pass — dados entram, previsão sai.

Para **aprender**, o modelo precisa de saber: em que direcção ajustar cada parâmetro?

Backpropagation: registar operações num grafo (forward), percorrê-lo ao contrário (backward).

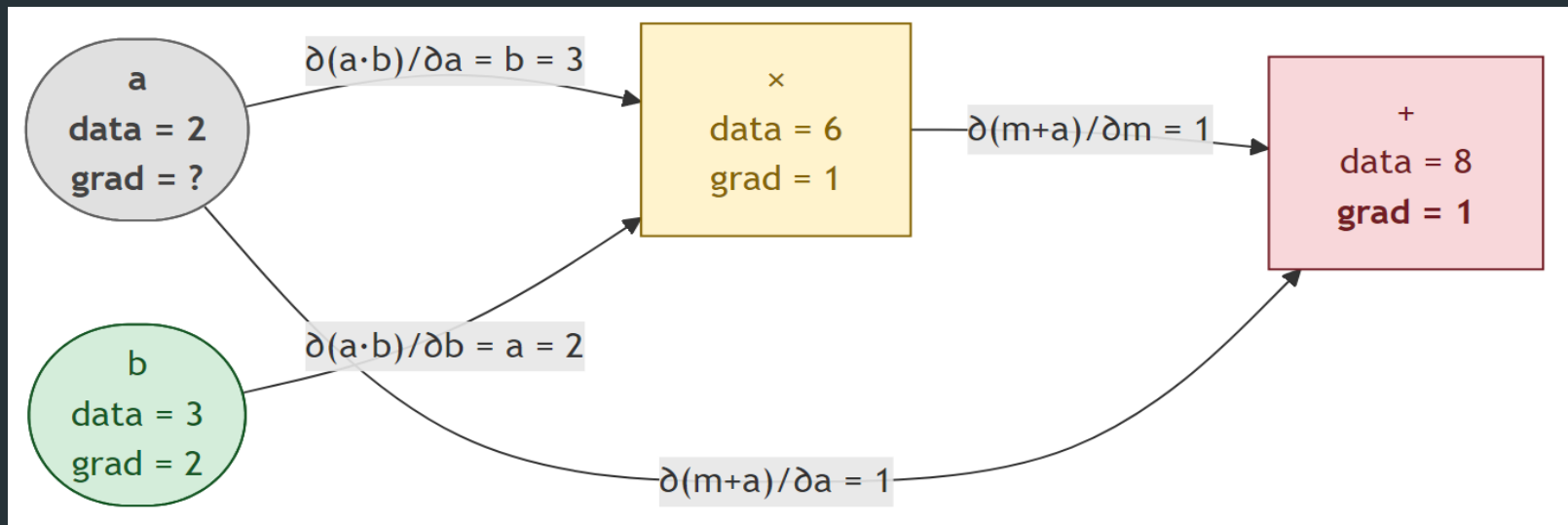


$c = a \cdot b + a$ — cada operação é um nó. O exercício 1 constrói este motor.

THINK: o backward pass

Começamos pelo output (grad = 1) e propagamos derivadas locais para trás:

◆ THINK



Dois caminhos chegam a **a**. Qual é o gradiente de **a**? → É o THINK do exercício 1 — façam no papel antes de codificar.

O que vos espera nos exercícios de hoje

Exercício 1 — Autograd Engine

TODOs 1–2 (forward): directos se tiverem feito as derivadas no papel

TODO 3 (backward): o mais difícil. Desenhem o grafo antes de codificar.

TODOs 4–5 (tokenizer): simples — aplicação directa do que acabaram de ver

Exercício 2 — Building Blocks

TODO 1 (linear): dot product em loop — se funciona, funciona

TODO 2 (softmax): parece simples, mas o Test 3 vai falhar. Leiam o comentário do teste com atenção.

TODO 3 (rmsnorm): se chegarem aqui hoje, estão bem encaminhados

Mini-Project 1: regras



Entrega

Semana 5

15% da nota final



Grupos

1 ou 2 alunos

Individual ou par



Entregável

6 ficheiros .py

+ explicações no ex6



Discussão

~5 min

por grupo, semana 5

Compreensão: **50%** Código correcto: **30%** Legibilidade + estrutura: **20%**

Código que não sabem explicar → nota baixa. **Compreensão > Volume.**

AI tutor: usar IA para pensar MAIS

Há um prompt de tutor socrático no Moodle. O tutor nunca dá código — faz perguntas, exige previsões, escala gradualmente.

✓ Bom uso

"Não percebo o chain rule aqui — porquê é que o gradiente acumula com +=?"

X Mau uso

"Implementa o backward pass para mim."

Usar IA para pensar **mais**, não para pensar **menos**.

Se o tutor vos der a resposta directamente, algo correu mal — recarreguem o prompt.

Mãos à obra

```
# 1. Descarreguem os exercícios do Moodle (pasta exercises/)  
# 2. Abram ex1_autograd.py no vosso editor
```

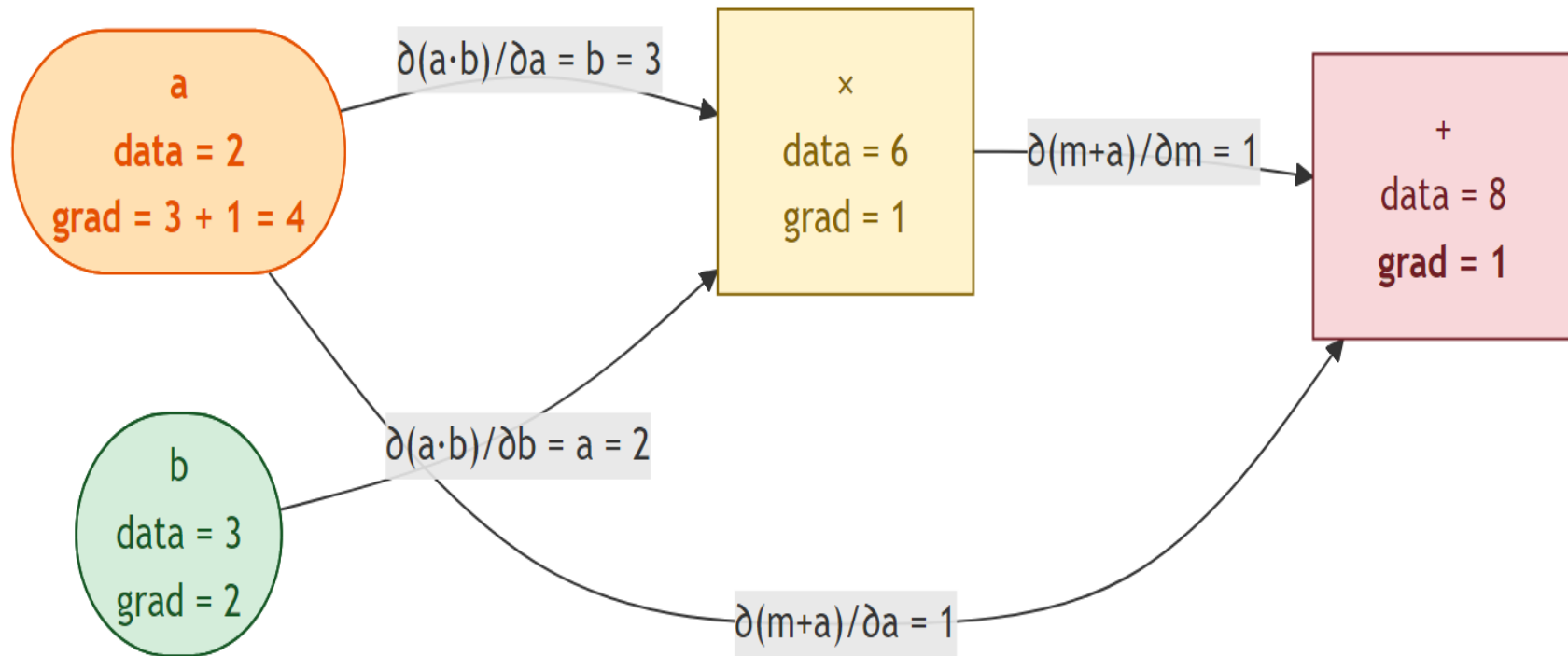
```
python ex1_autograd.py
```

Regra de ouro:

THINK no papel → **codificar** → **testar** → **REFLECT**

Prevejam antes de executar. Sempre.

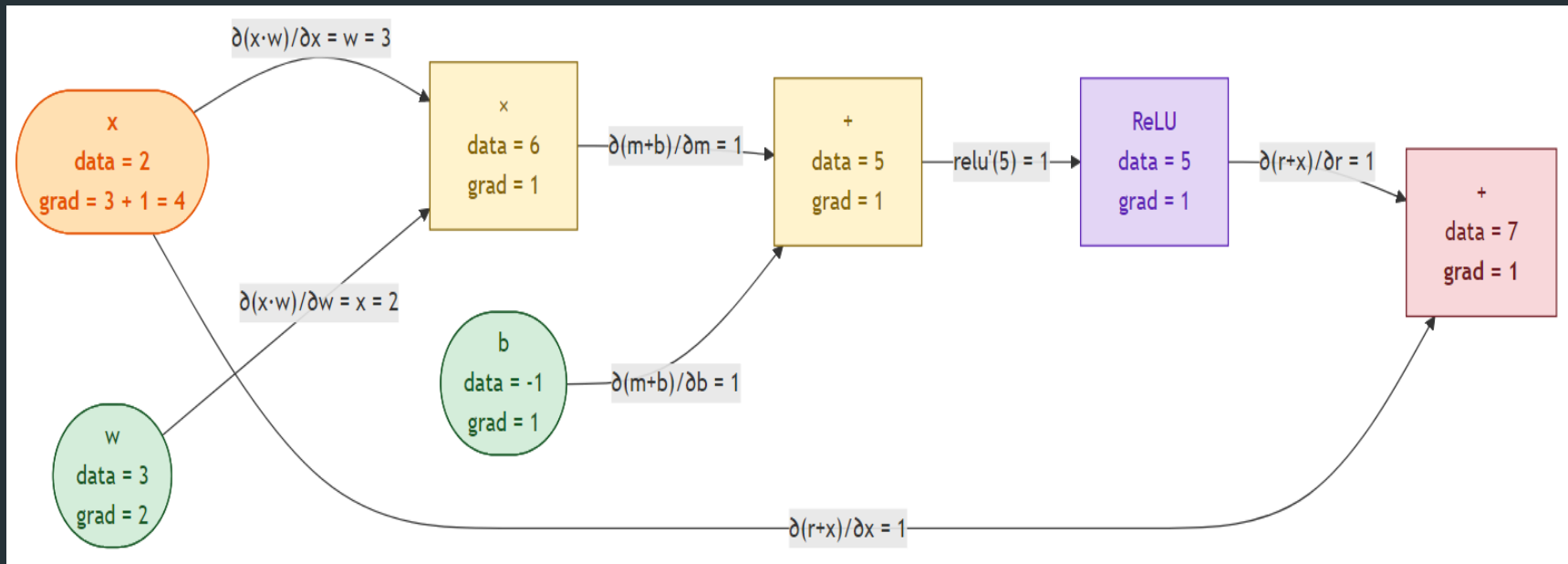
Referência: backward completo



$\text{grad}(a) = 3 + 1 = 4$ — o gradiente acumula (+) contribuições de cada caminho.

Referência: neurónio com residual

$z = \text{relu}(x \cdot w + b) + x$ — um neurónio com skip connection.



Se relu "morre" (input negativo $\rightarrow \text{relu}' = 0$), a residual connection ($+ x$) mantém o gradiente vivo. É por isso que as residual connections existem.